

Delybell Customer App - Project Handover Documentation

Prepared for developer handover Date: February 23, 2026 Project version: `3.0.0+1` (from `pubspec.yaml`)

Document Control

- Document type: Technical handover documentation
- Audience: Incoming developer / maintainer
- Application: Delybell Customer Mobile App (Flutter)
- Source reviewed: Current codebase in `delybell/`
- Purpose: Enable safe maintenance, debugging, and future enhancements

Table of Contents

1. Purpose of This Document
2. Executive Summary
3. High-Level Product Scope
4. Technology Stack
5. Project Structure (What Lives Where)
6. App Startup and Session Flow
7. Environment and Configuration
8. UI Navigation Model
9. Feature Documentation
10. State Management Pattern (BLoC)
11. Data Models and Mapping Notes
12. Backend API Catalog (Consolidated)
13. Build, Run, and Development Setup
14. Permissions and Device Capabilities
15. Logging and Error Handling
16. Testing Status (Important Handover Note)
17. Known Technical Risks and Maintenance Hotspots
18. Handover Checklist for the Next Developer
19. Suggested Refactor Plan (Practical, Low-Risk Sequence)
20. Troubleshooting Notes
21. Appendix: Important Files to Know First
22. Final Handover Summary

Feature Documentation Subsections

9.1 Authentication (Login / Signup / Forgot Password) 9.2 Dashboard (Home Tab) 9.3 Orders (Orders Tab) 9.4 Domestic Order Creation 9.5 Draft Placement / Pickup Scheduling 9.6 International Order Creation 9.7 Invoices (Invoices Tab) 9.8 Profile (Profile Tab) 9.9 Pickup Address Management 9.10 Nationality Picker 9.11 Barcode Scanning and Bluetooth Printing

1. Purpose of This Document

This document is the formal handover guide for the next developer who will maintain or extend the Delybell Customer mobile application.

It is intended to be:

- technically accurate (based on the current implementation)
- easy to understand for a new developer
- practical for day-to-day maintenance and debugging
- suitable for project ownership transfer

This document explains:

- what the application does
- how the codebase is organized
- how the major features work
- which backend APIs are currently used
- how to build and run the app
- key risks, limitations, and recommended next steps

2. Executive Summary

The Delybell Customer App is a Flutter-based mobile application used by customers to manage delivery operations.

Core capabilities currently implemented:

- authentication (user and corporate login)
- dashboard summary and operational metrics
- domestic order creation (multi-step flow)
- international order creation (multi-step flow)
- draft order review and placement scheduling
- order details and tracking timeline
- invoice listing (paid and unpaid)
- profile update and pickup address management
- barcode scanning and Bluetooth label printing

The application is functionally broad and useful as a production-ready foundation, but the following items require attention during ongoing maintenance:

- backend environment URL is hardcoded (currently staging)
- Android release signing is not configured for production
- auth/session data is stored in `SharedPreferences`
- several critical screens and BLoCs are very large (maintenance complexity)
- automated testing coverage is not yet established

3. High-Level Product Scope

Main user journeys

1. Sign in
2. View dashboard summary and recent/order counts
3. Create order (Domestic or International)
4. Review drafts and place pickups
5. Open order detail and tracking timeline
6. Print labels to Bluetooth printer (optional workflow)
7. View invoices and search them
8. Update profile and manage pickup addresses

Primary user types currently represented in app

- `User` login
- `Corporate` login

The code also adapts UI behavior using values returned in login response:

- `userId`
- `orderFlowTypes`
- `addressFormatType`
- `packageDescription`

4. Technology Stack

Framework / language

- Flutter
- Dart
- Material 3 UI (`useMaterial3: true`)

Key packages used

- `flutter\_bloc` / `bloc` / `equatable` (state management)

- ``http`` (REST API calls)
- ``shared_preferences`` (session + dashboard cache persistence)
- ``mobile_scanner`` (barcode/QR scanning)
- ``print_bluetooth_thermal`` (Bluetooth printer integration)
- ``shimmer`` (skeleton/splash shimmer visuals)
- ``url_launcher`` (open settings/app settings)

Local toolchain observed on this machine

- Flutter ``3.38.3``
- Dart ``3.10.1``

Note: The app's ``pubspec.yaml`` SDK constraint is ``^3.8.1``, which is compatible with the observed local Dart 3.10.x toolchain.

5. Project Structure (What Lives Where)

The project root contains a single Flutter app folder:

- ``delybell/``

Inside ``delybell/lib``, the codebase uses a mixed structure:

- ``core/`` -> shared constants, session manager, theme extension, error formatting
- ``login/`` and ``signup/`` -> auth UI + BLoC + repository (legacy-style module folders)
- ``dashboard/`` -> shared models and some older repositories/models
- ``features/`` -> newer feature-based organization for dashboard, orders, invoices, profile

Important note about architecture style

This codebase is in a transitional state:

- newer features are under ``lib/features/...`` with repository/domain/presentation separation
- older shared models and auth modules still live in top-level folders (``lib/dashboard``, ``lib/login``, ``lib/signup``)

This is not a problem functionally, but future refactoring should standardize one structure to reduce confusion.

6. App Startup and Session Flow

Entry point

- ``lib/main.dart``

Startup sequence:

1. Flutter bindings initialized

2. Status bar style configured
3. `DelybellApp` runs with `MultiRepositoryProvider`
4. `SplashGate` shown as home
5. `SplashGate` loads saved login from `SessionManager`
6. If session exists -> navigates to `DashboardPage`
7. If no session -> shows `LoginPage`

Repositories registered globally in `main.dart`

- `LoginRepository`
- `SignUpRepository`
- `DashboardRepository` (`DashboardRepositoryImpl`)
- `OrdersRepository` (`OrdersRepositoryImpl`)
- `InvoicesRepository` (`InvoicesRepositoryImpl`)

Session persistence

`SessionManager` stores login response JSON in `SharedPreferences` under key:

- `login_response`

Stored data includes auth token and user payload (serialized `LoginResponse`).

7. Environment and Configuration

Backend base URL (hardcoded)

`lib/core/static.dart` currently points to:

- `https://staging.api.delybell.com/`

Other URLs are commented out in code.

This means:

- there is no runtime environment switching
- no `.env` / flavor-based config currently implemented
- production handover should include a plan for environment management

Android app config (handover-critical)

From `android/app/build.gradle.kts`:

- namespace: `com.example.delybell`
- applicationId: `com.example.delybell` (placeholder package id)
- release build currently signs with debug signing config

From `AndroidManifest.xml`:

- ``android:usesCleartextTraffic="true"``
- permissions for internet/network/Bluetooth/location are declared

iOS config highlights

From ``ios/Runner/Info.plist``:

- app display name: ``Delybell``
- Bluetooth usage descriptions present
- Camera usage description present (for barcode scanning)

8. UI Navigation Model

The app uses direct ``Navigator`` + ``MaterialPageRoute`` navigation.

There is no centralized route table (``routes``, ``go_router``, etc.).

Main post-login navigation

``DashboardPage`` hosts a bottom navigation bar with 4 tabs:

- Home
- Orders
- Invoices
- Profile

Floating action button behavior

On the Home tab, a floating action button opens a bottom sheet with:

- Domestic Delivery
- International Delivery

9. Feature Documentation

9.1 Authentication (Login / Signup / Forgot Password)

Login

Files:

- ``lib/login/view/login_page.dart``
- ``lib/login/bloc/login_bloc.dart``
- ``lib/login/data/login_repository.dart``

Behavior:

- user chooses login type using segmented control:

- User
- Corporate
- form validates email/password
- on success:
- login response is saved to `SharedPreferences`
- app navigates to `DashboardPage`

Login API endpoints

- `POST user/login`
- `POST customer/corporate/login`

Forgot password

Files:

- `lib/login/view/forgot\_password\_page.dart`
- `lib/login/bloc/forgot\_password\_bloc.dart`

API:

- `POST user/forgot\_password`

UX:

- simple email input
- success snackbar + auto back navigation

Signup

Files:

- `lib/signup/view/sign\_up\_page.dart`
- `lib/signup/bloc/sign\_up\_bloc.dart`
- `lib/signup/data/sign\_up\_repository.dart`

API:

- `POST initiate\_registration`

Notes:

- supports segmented signup type (User / Corporate)
- collects first/last name, email, phone, password, confirm password
- country code dropdown included

9.2 Dashboard (Home Tab)

Files:

- `lib/features/dashboard/presentation/pages/dashboard_page.dart`
- `lib/features/dashboard/presentation/bloc/dashboard_bloc.dart`
- `lib/features/dashboard/data/dashboard_repository_impl.dart`
- shared model: `lib/dashboard/models/dashboard_summary.dart`

What it shows

Dashboard summary cards derived from backend summary response, such as:

- Live Orders
- Orders Today
- Pickup Today
- COD Amount (for COD-type users) or Delivered count (for others)

Data loading strategy

The dashboard repository caches the last summary in `SharedPreferences`:

- key: `dashboard_summary_cache`

On load:

- cached summary may be shown first
- fresh summary is requested next
- if fresh request fails and cache exists, UI can still show cached data

This improves perceived performance and resilience.

Dashboard API

- `GET customer/dashboard`

9.3 Orders (Orders Tab)

Primary files:

- `lib/features/dashboard/presentation/pages/dashboard_page.dart` (Orders tab UI is embedded here as `OrdersView`)
- `lib/features/orders/presentation/bloc/orders_bloc.dart`
- `lib/features/orders/data/orders_repository_impl.dart`
- models: `lib/dashboard/models/order_summary.dart`, `lib/dashboard/models/order_tracking.dart`

Tabs supported in Orders

- Draft
- In Progress

- Completed
- Canceled

Orders list capabilities

- search
- pull-to-refresh
- infinite scroll / pagination
- draft actions (delete, clear, future pickup, place)
- open draft details
- open order details
- cancel eligible orders (with reason support)

How list filtering works

In `OrdersBloc`, tabs map to backend behavior:

- Draft tab -> uses `customer/order/draft/list`
- Completed tab -> `filter\_by\_delivery\_status = 10`
- Canceled tab -> `filter\_by\_delivery\_status = 12`
- In Progress tab -> default `customer/order/list`

Order detail screen

File:

- `lib/features/orders/presentation/pages/order\_detail\_page.dart`

Capabilities:

- shows order summary, addresses, charges, package list
- decodes and displays barcode image (if backend returns base64 barcode)
- opens tracking screen
- opens Bluetooth label printing screen

Order tracking screen

File:

- `lib/features/orders/presentation/pages/order\_tracking\_page.dart`

Capabilities:

- loads tracking timeline
- renders status progression (ordered steps)
- retry on failure

- displays barcode + receiver/address details

Orders APIs (core list/detail/tracking/draft)

- `GET customer/order/list`
- `GET customer/order/tracking/{orderId}`
- `GET customer/order/draft/list`
- `POST customer/order/cancel`
- `POST customer/order/draft/initiate`
- `POST customer/order/draft/preview`
- `POST customer/order/draft/accept`
- `DELETE customer/order/draft/delete/{draftOrderId}`
- `DELETE customer/order/draft/clear`
- `GET customer/order/draft/details/{draftOrderId}`
- `GET customer/order/draft/place/preview`
- `POST customer/order/place`
- `PUT customer/order/draft/future\_pickup/{draftOrderId}`

9.4 Domestic Order Creation

Primary files:

- `lib/features/orders/presentation/pages/domestic\_order\_create\_page.dart`
- `lib/features/orders/presentation/bloc/domestic\_create\_bloc.dart`
- `lib/dashboard/data/domestic\_order\_repository.dart` (master data lookups)

This is one of the most complex parts of the app.

Workflow shape (multi-step)

The domestic flow is step-based and includes:

1. Service Type
2. Package Details
3. Delivery Details (or Pickup Details for return flow)
4. Draft Orders

Notable behavior observed

- can resume/edit an existing draft via `draftId`
- supports different order flow types using backend-provided `orderFlowTypes`
- reads address format / user details from saved login session
- supports barcode scanning for package IDs (`BarcodeScannerPage`)

- supports block/road/building pickers for Bahrain address lookups
- supports COD amount entry (based on package/order type behavior)

Master data used in domestic forms

- blocks
- roads (filtered by block)
- buildings (filtered by block + road)

These are loaded from master endpoints and reused in other places (such as profile address forms).

Domestic order-related lookup APIs

- `GET user/master/block/list`
- `GET user/master/road/list`
- `GET user/master/building/list`

9.5 Draft Placement / Pickup Scheduling

Files:

- `lib/features/orders/presentation/pages/draft\_place\_page.dart`
- `lib/features/orders/presentation/bloc/draft\_place\_bloc.dart`

Purpose:

- schedule pickup for draft orders before placing them
- preview charges/details
- choose pickup date and slot

Notable UX/business logic in code:

- time-based slot availability logic exists in UI (e.g., today/morning/evening availability)
- same-day behavior is treated differently from other service types

This logic should be validated against current business rules before future changes.

9.6 International Order Creation

Primary files:

- `lib/features/orders/presentation/pages/international\_order\_create\_page.dart`
- `lib/features/orders/presentation/bloc/international\_create\_bloc.dart`
- `lib/features/orders/data/international\_order\_repository.dart`

This is the largest single screen/workflow in the project.

Workflow shape (multi-step)

Observed step titles:

1. Shipment
2. Package Details
3. Shipper (rate selection)
4. From Address
5. To Address
6. Confirmation

Notable behavior

- loads defaults from saved login (user id, package description, address format type)
- requests origin country list on startup (default search uses Bahrain)
- loads pickup addresses and can prefill shipper address
- fetches courier/shipping rates
- groups and selects rate options
- initiates international draft and places order
- shows success screen after placement

International APIs

- ``GET user/master/country/list``
- ``GET customer/international/cities/list``
- ``POST customer/shipping/rates/all``
- ``POST customer/international/draft/initiate``
- ``GET customer/international/draft/details/{draftId}``
- ``POST customer/international/order/place``

9.7 Invoices (Invoices Tab)

Primary files:

- ``lib/features/invoices/presentation/pages/invoices_page.dart``
- ``lib/features/invoices/presentation/bloc/invoices_bloc.dart``
- ``lib/features/invoices/data/invoices_repository_impl.dart``

Features

- Paid / Unpaid tabs
- invoice search (by ID text)
- pagination / infinite scroll
- pull-to-refresh

- error and retry handling

Backend filtering

`InvoicesBloc` maps tabs to `payment\_status`:

- Paid -> `2`
- Unpaid -> `1`

Invoices API

- `GET customer/invoice/list`

9.8 Profile (Profile Tab)

Primary files:

- `lib/features/profile/presentation/pages/profile\_page.dart`
- `lib/features/profile/presentation/pages/profile\_edit\_page.dart`
- `lib/features/profile/presentation/bloc/profile\_edit\_bloc.dart`
- `lib/features/profile/data/profile\_repository\_impl.dart`

What profile supports

- view current user info
- edit profile fields
- save updated profile (and refresh saved session)
- manage pickup addresses
- logout
- open external URLs (via `url\_launcher` used in profile page)

Profile fields observed in edit flow

- first name / last name
- email
- phone (country code + number)
- company name
- company registration number
- VAT number
- address line / block field usage
- Arabic first/last name
- nationality
- package description

Profile update API

- `PUT customer/update\_profile`

On success:

- the returned updated user payload is converted back into `LoginResponse`
- session is re-saved via `SessionManager`

9.9 Pickup Address Management

Primary files:

- `lib/features/profile/presentation/pages/address\_list\_page.dart`
- `lib/features/profile/presentation/pages/address\_add\_page.dart`
- `lib/features/profile/presentation/pages/address\_form\_page.dart`
- `lib/features/profile/presentation/bloc/address\_list\_bloc.dart`
- `lib/features/profile/presentation/bloc/address\_form\_bloc.dart`
- `lib/features/profile/data/address\_repository\_impl.dart`

Address features

- list saved addresses
- add address
- edit address
- delete address
- mark address as primary

Address-related APIs

- `GET addresses/list`
- `POST addresses/create`
- `PUT addresses/update/{id}`
- `DELETE addresses/delete/{id}`
- `PUT addresses/update/primary/{id}`

Additional lookup APIs used by address form

- `GET user/master/block/list`
- `GET user/master/road/list`
- `GET user/master/building/list`

9.10 Nationality Picker

Files:

- `lib/features/profile/presentation/pages/nationality\_picker\_page.dart`

- ``lib/features/profile/presentation/bloc/nationality_bloc.dart``
- ``lib/features/profile/data/nationality_repository_impl.dart``

API:

- ``GET user/master/nationality/list``

9.11 Barcode Scanning and Bluetooth Printing

Barcode scanning

File:

- ``lib/features/orders/presentation/pages/barcode_scanner_page.dart``

Used in domestic order package entry to capture package identifiers.

Package dependency:

- ``mobile_scanner``

Bluetooth label printing

File:

- ``lib/features/orders/presentation/pages/bluetooth_print_page.dart``

Capabilities:

- checks Bluetooth permissions
- lists paired devices
- connects to selected printer
- sends TSPL bytes for label printing
- opens device settings if permission/Bluetooth is unavailable

Package dependencies:

- ``print_bluetooth_thermal``
- ``url_launcher``

10. State Management Pattern (BLoC)

The app heavily uses BLoC with explicit events and immutable state.

Common pattern per feature:

- ``..._event.dart``
- ``..._state.dart``
- ``..._bloc.dart``

Advantages in current code:

- UI state transitions are mostly explicit
- async API handling is organized per feature
- easier to add loading/error states

Current downside:

- some BLoCs have grown very large (especially order creation)
- business rules and UI flow rules are mixed in the same files

11. Data Models and Mapping Notes

The app contains defensive JSON parsing to tolerate backend response variations.

Examples:

- login user model supports alternate keys (`firstName`, `first\_name`, etc.)
- order list parsing supports multiple response shapes (list vs paginated object)
- address formatting composes human-readable lines from block/road/building details

This is a strength for compatibility, but it also indicates backend response contracts may not be fully stable/consistent.

12. Backend API Catalog (Consolidated)

This is the API surface used by the current app (from repository and related BLoC code).

Authentication

- `POST user/login`
- `POST customer/corporate/login`
- `POST user/forgot\_password`
- `POST initiate\_registration`

Dashboard

- `GET customer/dashboard`

Orders / Draft Orders / Tracking

- `GET customer/order/list`
- `GET customer/order/tracking/{orderId}`
- `GET customer/order/draft/list`
- `GET customer/order/draft/details/{draftOrderId}`
- `POST customer/order/draft/initiate`
- `POST customer/order/draft/preview`

- `POST customer/order/draft/accept`
- `DELETE customer/order/draft/delete/{draftOrderId}`
- `DELETE customer/order/draft/clear`
- `GET customer/order/draft/place/preview`
- `POST customer/order/place`
- `PUT customer/order/draft/future\_pickup/{draftOrderId}`
- `POST customer/order/cancel`

International Shipping

- `GET user/master/country/list`
- `GET customer/international/cities/list`
- `POST customer/shipping/rates/all`
- `POST customer/international/draft/initiate`
- `GET customer/international/draft/details/{draftId}`
- `POST customer/international/order/place`

Profile / Address / Reference Data

- `PUT customer/update\_profile`
- `GET addresses/list`
- `POST addresses/create`
- `PUT addresses/update/{id}`
- `DELETE addresses/delete/{id}`
- `PUT addresses/update/primary/{id}`
- `GET user/master/nationality/list`
- `GET user/master/block/list`
- `GET user/master/road/list`
- `GET user/master/building/list`

Invoices

- `GET customer/invoice/list`

13. Build, Run, and Development Setup

13.1 Prerequisites

- Flutter SDK (stable; version used locally during review: `3.38.3`)
- Dart SDK compatible with Flutter version
- Android Studio / Xcode (depending on platform)
- device/emulator with Bluetooth + camera if testing printer/scanner flows

13.2 Setup commands

From `delybell`:

```
flutter pub get  
flutter run
```

Optional:

```
flutter analyze  
flutter test
```

13.3 Build commands (examples)

```
flutter build apk  
flutter build appbundle  
flutter build ios
```

Important before release:

- configure real Android `applicationId`
- configure release signing (currently debug signing is used)
- confirm production `baseUrl`

14. Permissions and Device Capabilities

Android permissions declared

- Internet / network state
- Bluetooth (classic + BLE + scan/connect/advertise)
- Fine/coarse location

Reason (in app functionality):

- Bluetooth printer discovery/connection often requires Bluetooth + location-related permissions on Android
- API communication requires internet access

iOS permissions declared

- Bluetooth usage description
- Camera usage description

Reason:

- Bluetooth printing
- barcode/QR scanning

15. Logging and Error Handling

Logging

Many repository implementations log:

- request URLs
- status codes
- response bodies (sometimes preview-truncated, sometimes full)
- request payloads

This is useful during development but risky in production because logs may contain:

- tokens
- customer personal data
- addresses
- phone numbers

Recommended next step:

- replace ``print`/`debugPrint`` with a configurable logger
- disable sensitive logs in release builds

Error handling

``ErrorUtils.friendly(...)`` provides user-friendly messages for common errors:

- network unavailable
- timeout
- server unavailable
- session expired

This improves UX and should be retained.

16. Testing Status (Important Handover Note)

Current state:

- automated test coverage is effectively missing
- ``test/widget_test.dart`` is still the default Flutter counter sample test and does not match this app

Impact:

- regressions are likely when changing large order flows
- manual QA is currently required for every release

Recommended minimum test plan to add first:

1. repository JSON parsing tests (login, orders, invoices)
2. BLoC tests for ``OrdersBloc``, ``InvoicesBloc``, ``DashboardBloc``
3. smoke widget tests for login and dashboard tab rendering

4. golden/snapshot tests only after UI stabilizes

17. Known Technical Risks and Maintenance Hotspots

These are the most important items for the next developer to know.

1. Hardcoded environment URL

Risk:

- easy to accidentally build against staging

Recommendation:

- introduce environment flavors (`dev`, `staging`, `prod`)

2. Android release signing still uses debug config

Risk:

- release build process is not production-ready

Recommendation:

- configure proper keystore and signing config before release

3. Session token stored in `SharedPreferences`

Risk:

- not ideal for sensitive auth tokens

Recommendation:

- move to secure storage (for example `flutter\_secure\_storage`)

4. Very large files (high maintenance complexity)

Examples observed:

- `international\_order\_create\_page.dart` (~1900+ lines)
- `domestic\_order\_create\_page.dart` (~1600+ lines)
- `dashboard\_page.dart` (~1400+ lines)
- `domestic\_create\_bloc.dart` (~800+ lines)
- `international\_create\_bloc.dart` (~800+ lines)

Risk:

- slow onboarding
- harder debugging
- merge conflicts

Recommendation:

- split into smaller widgets/services/validators/payload mappers

5. Mixed architecture styles

Risk:

- feature ownership boundaries are less clear

Recommendation:

- gradually migrate legacy folders into `features/` or establish a stable shared-layer policy

6. Production logging concerns

Risk:

- sensitive payloads may leak into logs

Recommendation:

- redact payloads and disable verbose network logs in release builds

18. Handover Checklist for the Next Developer

Use this as the first-week checklist.

Access and environment

- obtain backend API credentials for staging and production
- confirm expected production base URL
- confirm app package IDs / bundle IDs
- obtain Android keystore and iOS signing/provisioning access

Technical validation

- run app on Android and iOS
- verify login (user + corporate)
- verify dashboard loads with real account
- test domestic order draft flow end-to-end
- test international order creation with rate retrieval
- test order tracking screen
- test invoice list/search
- test profile update and address CRUD
- test barcode scanner and Bluetooth printer on device

Codebase improvements to prioritize

- introduce environment/flavor config
- replace debug release signing
- remove default widget test, add real tests
- split large order screens and BLoCs
- review and reduce sensitive network logging

19. Suggested Refactor Plan (Practical, Low-Risk Sequence)

If continuing development, this order is recommended:

1. Environment configuration + release signing fixes
2. Logging sanitization and token handling review
3. Add repository/BLoC unit tests for current behavior
4. Extract domestic/international form sections into smaller widgets
5. Move payload building/validation into dedicated services
6. Standardize module structure across legacy and feature folders

This sequence reduces release risk first, then improves maintainability.

20. Troubleshooting Notes

Login works but dashboard fails

Check:

- token validity
- current `Static.baseUrl`` target
- backend environment availability
- printed repository response logs (dev only)

Bluetooth printer not discovered

Check:

- device Bluetooth enabled
- Android permissions granted (Bluetooth + location-related)
- printer paired at OS level
- app is running on a real device (not emulator)

Barcode scanner not working

Check:

- camera permission on device
- app installed with camera permission accepted

- real device testing (not all emulators provide camera behavior)

Order lists look wrong across tabs

Check:

- `OrdersBloc` tab mapping to endpoint/filter values
- backend delivery status IDs (`10`, `12`) still valid
- dashboard summary cache not being mistaken for list data (counts are cached separately)

21. Appendix: Important Files to Know First

Read these files first when onboarding:

- `lib/main.dart` -> app startup, dependency injection, splash/session gate
- `lib/core/static.dart` -> backend base URL
- `lib/login/models/login\_response.dart` -> session/user payload structure
- `lib/features/dashboard/presentation/pages/dashboard\_page.dart` -> main navigation + dashboard/orders/invoices wrappers
- `lib/features/orders/data/orders\_repository\_impl.dart` -> order/draft/tracking API integration
- `lib/features/orders/presentation/bloc/domestic\_create\_bloc.dart` -> domestic order business flow
- `lib/features/orders/presentation/bloc/international\_create\_bloc.dart` -> international order business flow
- `lib/features/orders/presentation/pages/domestic\_order\_create\_page.dart` -> domestic form UX
- `lib/features/orders/presentation/pages/international\_order\_create\_page.dart` -> international form UX
- `lib/features/profile/data/address\_repository\_impl.dart` -> pickup address CRUD APIs

22. Final Handover Summary

This app already implements a broad customer logistics workflow and is a strong base for continued development.

The most important next steps are not feature additions; they are operational hardening and maintainability improvements:

- environment configuration
- release signing setup
- test coverage
- large-file decomposition
- safer logging/session storage

Once these are addressed, future feature work will be faster and lower risk.